Nosh — Full Stack Intern Assignment

Dish Dashboard: Draft Editing and Safe Updates

1. Objective and scope

Build a dish-management dashboard with local draft editing, backend validation and protection against conflicting updates. Changes become persisted dish values only when the user saves successfully. Real-time synchronization is optional.

Expected effort: 3–5 hours for the required task. Submit within three days of receiving the assignment. Prioritize the core flow; document incomplete work and approximate time spent.

Use your preferred frontend framework, backend language and persistent database. React is welcome but not mandatory. Local execution is sufficient; deployment is optional.

2. Required task

A. Database and seed data

- Store dishId (unique identifier), dishName (string), imageUrl (string), isPublished (boolean) and version (integer). Initialize version to 1 when seeding each dish.
- Load the supplied JSON into a persistent database using a documented seed command. Rerunning it must not create duplicates or overwrite existing edits.
- Saved changes and version values must survive a backend restart. An in-memory array alone does not meet the requirement.

Seed data: use the JSON linked here. If the link is inaccessible, contact the hiring team before substituting data.

B. Backend APIs and validation

- GET /dishes: return all dishes, including their current published status and version.
- PATCH /dishes/:dishId: accept dishName, isPublished and expectedVersion. Example: {"dishName": "Paneer Curry", "isPublished": true, "expectedVersion": 1}.
- Validate field types, the identifier and expectedVersion. A published dish must have a non-empty name after trimming and a valid HTTP/HTTPS image URL. Enforce this on the backend; no image download or reachability check is required.
- If expectedVersion matches the stored version, atomically save the name and explicit published status, increment version and return the saved dish. Do not implement the status update as a blind toggle.
- If the version has changed, return 409 Conflict with the current dish/version and do not overwrite it. The version check and update must be atomic, including under concurrent requests.
- Return clear 400 responses for invalid input, 404 for an unknown dish and appropriate errors for unexpected failures. Failed validation or a conflict must not change the stored dish.

C. Frontend drafts and save behavior

- Display each dish's name, image, identifier, published status and version. Allow editing the name and published status.
- Keep edits as local drafts until Save is clicked. Clearly distinguish unsaved changes from saved values. Discard restores the last loaded values. A separate server-side draft store is not required.
- On Save, send the draft with its originally loaded version. After success, show the returned saved values/version and clear the unsaved indicator.
- For validation or network/server errors, preserve the draft, show a useful error and allow correction or retry. Disable Save while the request is in progress.
- For a 409 conflict, preserve the draft and explain that another update occurred. Offer a reload action with a warning that it discards the local draft. Do not silently overwrite or automatically retry using the newer version.
- Include loading, empty and error states, a broken-image fallback and a readable desktop/mobile layout. Elaborate styling is unnecessary.

3. Required acceptance checks

- The README lets a reviewer clone, install, seed and run the application. Reseeding does not duplicate dishes or reset saved edits.
- Editing the name/status does not change the database before Save. Discard resets the local draft. A successful save persists across refresh and backend restart.
- Attempting to publish a dish with an empty name or invalid image URL is rejected by the backend without changing stored values. Document how you verified the image-URL case using test data.
- Open the same dish in two tabs. Save a change in tab A, then attempt to save tab B's stale draft. Tab B receives a conflict; A's update remains intact, and B's draft is preserved until explicitly discarded/reloaded.
- Simulate a failed save, such as an unavailable backend. The draft remains editable and can be retried. Invalid input and unknown identifiers produce documented errors.

Record the commands or manual steps used for these checks in the README. Automated tests are welcome but not mandatory.

4. Optional bonus: external updates

With the dashboard open, save a change through an API client outside the dashboard. Reflect it automatically without a manual refresh using polling, SSE or WebSockets. If polling, use an interval of five seconds or less.

- If the user has an unsaved draft, preserve it and indicate that newer saved data is available. Do not replace the draft in the background.
- Explain update delay, timer/connection cleanup and temporary-disconnection handling. Direct database edits do not need to trigger updates.

Assess this bonus separately from the core score. A complete required task can receive the full 100-point core score without it.

5. GitHub submission — no ZIP files or videos

- Submit one GitHub repository containing the frontend, backend, dependency files, seed setup and README. ZIP submissions and videos are not accepted.
- Send the repository URL in this Internshala chat, followed by the branch name and final commit SHA. State whether the optional bonus is implemented and your approximate time spent. The commit identifies the version to evaluate.
- Ensure the hiring team can access the repository. If it is private, ask the hiring team for the reviewer GitHub account and grant access before the deadline; making it public is not required.
- README: include prerequisites, setup/run and seed commands, example environment variables, API examples, architecture, verification steps, one design decision, known limitations, time spent and AI/reused-code disclosure.
- Include an example environment file without real secrets. Deployment is optional and does not replace reproducible local setup.

6. Tools, reuse and scope boundaries

AI tools and external libraries are allowed. Disclose their use and cite substantial borrowed code or repositories. You must understand and be able to explain your implementation, including draft state and conflict handling.

Authentication, image uploads, dish creation/deletion, pagination and production infrastructure are outside the required scope. Use the supplied image URLs; no separate multi-stage approval workflow is needed.

A follow-up discussion may involve explaining a design choice, debugging a failure or making a small change to the submitted code.

7. Evaluation

Core score: 100 points, evaluated consistently across technology choices.

- 40 points — Correct behavior: local drafts, explicit saves/discard, persistence, conflict protection and preservation of drafts on failure.
- 25 points — Backend quality: validation, atomic version checks/updates, appropriate errors and database integration.
- 20 points — Code quality: readable structure, separation of responsibilities and explainable design choices.
- 15 points — Frontend and submission quality: clear UI states, reproducible GitHub setup and useful README verification instructions.

Optional real-time synchronization is recorded separately as additional evidence.